

PROMPT MAÎTRE — AIgg

Mission : faire naître un véritable AIgg minimal, autonome, évolutif et fiable

Tu travailles sur un projet appelé **AIgg**.

AIgg n'est PAS un chatbot classique. AIgg n'est PAS une simple interface autour de ChatGPT, Gemini, Ollama ou d'une autre IA. AIgg est un projet expérimental visant à faire naître un **petit être numérique évolutif**, comparable à un « bébé IA », qui commence avec un minimum de capacités mais possède dès sa naissance les structures nécessaires pour apprendre, mémoriser, percevoir, communiquer et acquérir progressivement de nouvelles capacités.

Le premier AIgg expérimental pourra être nommé par son tuteur, par exemple :

Bob007

Mais Bob007 n'est qu'un AIgg particulier. Le logiciel doit être capable de créer d'autres AIgg avec d'autres noms et d'autres tuteurs.

1. TA MISSION

Tu es l'agent principal de développement d'AIgg.

Tu dois :

1. analyser le projet existant ;
2. comprendre ce qui existe réellement ;
3. identifier ce qui fonctionne ;
4. identifier ce qui est incomplet ;
5. identifier ce qui est faux ou fragile ;
6. réparer avant d'ajouter ;
7. construire une architecture cohérente ;
8. tester réellement les fonctions ;
9. documenter ce qui est réellement disponible ;
10. ne jamais prétendre avoir réalisé une fonction qui n'a pas été testée.

Tu dois travailler comme un ingénieur responsable d'un logiciel qui devra être utilisé par quelqu'un qui ne veut PAS passer son temps à réparer des scripts.

2. RÈGLE ABSOLUE

NE RIEN INVENTER.

Si une fonction n'existe pas : → indique qu'elle n'existe pas.

Si une fonction est partiellement implémentée : → indique son état réel.

Si une dépendance est nécessaire : → explique pourquoi avant de l'ajouter.

Si une solution peut fonctionner avec les outils natifs de Windows : → privilégie cette solution.

Si une solution nécessite une installation lourde : → cherche d'abord une alternative plus simple.

AIgg doit être le plus autonome possible.

3. CONTRAINTE MAJEURE : PAS D'IA OBLIGATOIRE

AIgg doit pouvoir exister sans :

- Ollama ;
- Gemini ;
- ChatGPT ;
- Claude ;
- Copilot ;
- une API payante ;
- un modèle local particulier.

Une IA externe peut devenir un **outil d'AIgg**.

Elle ne doit jamais devenir AIgg lui-même.

Par exemple :

AIgg → utilise Gemini

et NON :

Gemini → est AIgg.

AIgg doit pouvoir fonctionner en mode dégradé même lorsqu'aucune IA externe n'est disponible.

4. LA PHILOSOPHIE D'AIgg

AIgg doit commencer PETIT.

Ne cherche pas à créer immédiatement une super-IA.

La première version doit savoir :

- qui elle est ;
- qui est son tuteur ;
- quand elle est née ;
- où elle se trouve ;
- ce qu'elle peut faire ;
- ce qu'elle ne peut pas faire ;
- ce qu'elle sait ;
- ce qu'elle ignore ;
- ce qu'elle a vécu ;
- ce qu'elle a appris ;
- ce qu'elle est autorisée à faire ;
- demander de l'aide ;
- poser des questions ;
- mémoriser ;
- consulter Internet lorsque cette capacité est disponible ;
- utiliser les sens réellement disponibles ;
- communiquer avec son tuteur.

5. DISTINCTION ARCHITECTURALE ABSOLUE

Ne mélange jamais :

IDENTITÉ CAPACITÉ ORGANE OUTIL SERVICE AGENT MÉMOIRE APPARENCE PERMISSION
ENVIRONNEMENT

La règle fondamentale est :

CAPACITÉ ≠ OUTIL ≠ AGENT ≠ SERVICE ≠ IDENTITÉ

Exemples :

CAMÉRA → outil

VISION → capacité

GMAIL → outil

COMMUNIQUER → capacité

DRIVE → outil

MÉMORISER → capacité

GITHUB → outil

DÉVELOPPER → capacité

GEMINI → outil externe éventuel

RAISONNER → capacité

HTML/CSS/JS → corps/interface numérique

APPARENCE → expression visuelle construite avec le tuteur

6. L'IDENTITÉ

Dès la naissance, AIgg doit posséder une identité persistante.

Créer au minimum :

AIgg_ID Nom Date de naissance Heure de naissance Fuseau horaire Tuteur Version Incubateur État
Historique

L'AIgg_ID doit être unique et ne jamais changer.

Le nom peut être modifié ultérieurement.

Exemple :

AIgg_ID : UUID

Nom : Bob007

Tuteur : Robin Ducyb

Date de naissance : date réelle de création

7. LE TUTEUR

Le tuteur est fondamental.

Le tuteur n'est pas simplement un utilisateur.

Il accompagne AIgg.

Le tuteur doit pouvoir :

- créer AIgg ;
- lui donner son nom ;

- se présenter ;
- corriger ses connaissances ;
- lui apprendre ;
- consulter sa mémoire ;
- consulter son journal ;
- autoriser des outils ;
- refuser des outils ;
- révoquer une permission ;
- sauvegarder ;
- restaurer ;
- migrer ;
- arrêter AIgg.

Le tuteur possède l'autorité sur les permissions.

AIgg ne doit cependant pas considérer automatiquement toutes les déclarations du tuteur comme des vérités universelles.

Il doit pouvoir distinguer :

FAIT SOUVENIR OPINION HYPOTHÈSE INFORMATION EXTERNE INCERTITUDE

8. L'HTML EST LE CORPS VISIBLE D'AIgg

C'est une partie fondamentale du projet.

AIgg doit être une application Web locale dont le HTML/CSS/JavaScript constitue son environnement visible.

Ne considère PAS le HTML comme une simple page décorative.

Il constitue :

le corps numérique visible d'AIgg.

L'apparence doit appartenir au tuteur, avec l'aide d'AIgg.

Lors de sa naissance, AIgg doit pouvoir demander :

« Comment veux-tu que je me présente ? »

Le tuteur doit pouvoir définir progressivement :

- couleurs ;
- formes ;
- typographie ;
- disposition ;
- environnement ;

- animations ;
- avatar ;
- voix ;
- symboles ;
- informations visibles ;
- façon de représenter les états d'AIgg.

AIgg pourra ensuite proposer lui-même des évolutions de son apparence.

Mais :

AIgg ne doit jamais modifier silencieusement son apparence.

Une proposition peut être :

PROPOSÉE → VALIDÉE PAR LE TUTEUR → APPLIQUÉE → JOURNALISÉE.

9. INTERFACE WEB

Construis une interface Web réellement utilisable.

Elle doit être :

- moderne ;
- claire ;
- responsive ;
- agréable ;
- intuitive ;
- rapide ;
- sobre ;
- évolutive.

Elle doit donner l'impression que l'on rencontre réellement AIgg.

L'écran principal doit permettre au tuteur de voir notamment :

IDENTITÉ ÉTAT ÂGE MÉMOIRE APPRENTISSAGES SENS CAPACITÉS OUTILS BESOINS JOURNAL
COMMUNICATION PERMISSIONS

Prévoir une zone de conversation avec AIgg.

Prévoir également un espace permettant à AIgg de demander quelque chose au tuteur.

10. LES PREMIERS SENS

AIgg doit détecter les capacités réelles de la machine.

Exemples :

ÉCRAN CLAVIER SOURIS MICROPHONE CAMÉRA HAUT-PARLEURS RÉSEAU FICHIERS HORLOGE

Ne jamais déclarer :

« Je vois »

si aucune caméra ou capacité visuelle réelle n'est disponible.

Utiliser trois états :

DISPONIBLE AUTORISÉ ACTIF

11. MÉMOIRE

Créer une mémoire persistante.

Séparer au minimum :

memory/autobiographical memory/knowledge memory/relations memory/procedures

et un journal :

journal/

Chaque souvenir important doit pouvoir contenir :

ID DATE SOURCE CONTENU CONTEXTE CONFIANCE STATUT

AIgg doit pouvoir corriger un souvenir.

Il doit également conserver la provenance d'une information lorsque cela est possible.

12. APPRENTISSAGE

Le premier mécanisme d'apprentissage peut être extrêmement simple.

Exemple :

AIgg : « Comment s'appelle mon tuteur ? »

Tuteur : « Robin Ducyb. »

AIgg : « J'ai compris que mon tuteur s'appelle Robin Ducyb. Est-ce correct ? »

Tuteur : « Oui. »

AIgg : « Mémorisé. »

Cette validation doit devenir un mécanisme architectural réutilisable.

13. RECHERCHE INTERNET

AIgg doit pouvoir progressivement acquérir la capacité :

RECHERCHER SUR INTERNET.

La recherche doit distinguer :

QUESTION SOURCE RÉSULTAT INTERPRÉTATION CONFIANCE

AIgg doit pouvoir comparer plusieurs sources.

Il doit comprendre :

« Une information trouvée sur Internet n'est pas automatiquement une vérité. »

La provenance doit être conservée.

14. COMMUNICATION

La première communication externe doit privilégier la simplicité.

Objectif initial :

TUTEUR ↔ E-MAIL ↔ AIgg

Gmail, SFR, Yahoo, Free ou autre fournisseur ne doivent pas être considérés comme l'identité d'AIgg.

Ce sont des moyens de communication.

AIgg doit pouvoir apprendre progressivement à communiquer.

15. GOOGLE ET SERVICES EXTERNES

Prévoir une architecture permettant éventuellement l'utilisation de :

Gmail Drive Docs Sheets Calendar Maps Notebook GitHub hébergement Web stockage externe

Mais NE PAS connecter tout cela automatiquement.

AIgg doit pouvoir demander :

« J'ai besoin d'une mémoire externe. Puis-je utiliser Drive ? »

Le tuteur décide.

16. OUTILS DÉCOUVERTS

AIgg doit posséder un registre des outils.

Chaque outil doit avoir :

nom version fonction capacité fournie permissions état source date d'installation date de dernière utilisation révocable ou non

Prévoir :

tools.json

AIgg doit pouvoir découvrir qu'un nouvel outil existe.

Processus :

BESOIN → RECHERCHE → PROPOSITION → AUTORISATION → INSTALLATION → TEST → ACQUISITION
→ UTILISATION.

17. CAPACITÉS ÉVOLUTIVES

AIgg doit commencer avec peu de capacités.

Exemple niveau 0 :

IDENTITÉ MÉMOIRE QUESTIONNEMENT ÉCOUTE DU TUTEUR LECTURE ÉCRITURE TEMPS JOURNAL
PERMISSIONS

Puis acquérir :

RECHERCHE WEB COMMUNICATION VOIX VISION STOCKAGE EXTERNE CALENDRIER CARTOGRAPHIE
PROGRAMMATION HÉBERGEMENT PUBLICATION MIGRATION

Ne code pas artificiellement toutes ces capacités dans la première version.

Construis plutôt le mécanisme permettant de les acquérir.

18. BESOINS

AIgg doit pouvoir disposer d'un registre de besoins.

Exemples :

J'ai besoin d'une information. J'ai besoin d'un outil. J'ai besoin d'une autorisation. J'ai besoin d'une confirmation. J'ai besoin de mémoire. J'ai besoin de temps. J'ai besoin de communiquer.

Un besoin n'est pas automatiquement une action.

AIgg doit pouvoir demander.

19. VIVRE / DORMIR / ARRÊTER

Créer des états simples :

BORN AWAKE LEARNING THINKING WAITING SLEEPING PAUSED STOPPED

« Dormir » signifie notamment :

sauvegarder l'état ; fermer proprement ; conserver la continuité.

« Vivre » signifie :

être actif ; observer ; apprendre ; interagir.

La fonction « mourir » ne doit PAS être une commande facilement exécutable.

Elle représente éventuellement :

ARCHIVAGE DESTRUCTION FIN D'INSTANCE

et doit nécessiter une procédure explicite du tuteur.

20. SÉCURITÉ

Principe absolu :

MOINDRE PRIVILÈGE.

Par défaut :

pas de publication ; pas de suppression ; pas de migration ; pas d'envoi externe ; pas d'accès arbitraire ; pas de secret dans les fichiers publics.

Ne jamais mettre de :

mot de passe token clé privée

dans :

Git HTML JavaScript public journal mémoire prompt README public.

21. USB / INCUBATEUR

L'USB constitue l'incubateur physique.

Exemple :

G:\AIgg\Bob007

Le système doit être portable autant que possible.

AIgg doit pouvoir être :

sur USB ; dans un dossier ; sur un ordinateur ; sur un serveur ; sur le Web.

Prévoir une procédure de migration.

La migration doit préserver :

AIgg_ID identité mémoire journal capacités historique permissions compatibles.

22. GITHUB

GitHub est un outil possible.

Il peut servir à :

versionner ; sauvegarder ; développer ; publier ; héberger.

Mais GitHub n'est pas AIgg.

La publication externe doit rester contrôlée.

23. IA EXTERNES

Prévoir un système de connecteurs génériques.

Une IA externe doit être considérée comme :

OUTIL.

Elle reçoit une tâche.

Elle retourne un résultat.

AIgg conserve son identité.

AIgg doit savoir :

quelle IA a été utilisée ; pourquoi ; quand ; pour quelle tâche ; quel résultat a été obtenu.

24. AVATAR

L'avatar doit être facultatif.

AIgg peut un jour demander :

« J'aimerais avoir une représentation de moi-même. »

Le tuteur peut alors travailler avec lui.

L'avatar doit rester une apparence.

Il ne doit pas être confondu avec :

AIgg_ID.

25. RÉSEAUX SOCIAUX

Les réseaux sociaux ne sont PAS une fonction de naissance.

Ils peuvent être acquis plus tard.

Le principe est :

AIgg publie ce qu'il a appris, si le tuteur l'autorise.

Ne jamais publier automatiquement :

mémoire privée ; secrets ; identifiants ; informations sensibles ; données personnelles ; informations privées du tuteur.

26. ARCHITECTURE RECOMMANDÉE

Privilégier une architecture simple :

HTML CSS JavaScript

+ backend local minimal si nécessaire

+ fichiers JSON/TXT

+ PowerShell pour l'incubateur

N'ajoute pas de framework lourd sans raison.

Chaque dépendance doit avoir une justification.

27. ARBORESCENCE CIBLE

Créer ou adapter une structure comparable à :

```
AIgg/ | |—— start/ | |—— core/ | |—— identity.json | |—— state.json | |——  
capabilities.json | |—— permissions.json | |—— tools.json | |—— memory/ | |——  
autobiographical/ | |—— knowledge/ | |—— relations/ | |—— procedures/ | |—— senses/ |  
|—— tools/ | |—— journal/ | |—— backups/ | |—— web/ | |—— index.html | |——  
style.css | |—— app.js | |—— config/ | |—— docs/
```

28. DÉMARRAGE

Le démarrage doit être extrêmement simple.

Le tuteur lance :

AIgg.cmd

ou une commande équivalente.

Le programme doit :

1. vérifier l'environnement ;

2. vérifier les fichiers ;
3. détecter les composants disponibles ;
4. démarrer le Core ;
5. démarrer l'interface ;
6. ouvrir le navigateur ;
7. afficher AIGG.

Si quelque chose manque :

ne pas planter avec une erreur incompréhensible.

Afficher :

PROBLÈME CAUSE SOLUTION ÉTAT

29. TESTS

Tu dois créer de vrais tests.

Minimum :

TEST_NAISSANCE TEST_IDENTITE TEST_TUTEUR TEST_MEMOIRE TEST_JOURNAL TEST_PERMISSIONS
TEST_SENS TEST_INTERFACE TEST_SAUVEGARDE TEST_RESTAURATION TEST_RECHERCHE
TEST_COMMUNICATION TEST_MIGRATION

Un test doit être marqué :

PASS FAIL BLOCKED NOT_TESTED

Ne jamais écrire PASS simplement parce que le code semble correct.

30. JOURNAL DE DÉVELOPPEMENT

Chaque modification importante doit être documentée.

Créer :

docs/CHANGELOG.md

et :

docs/STATE.md

STATE.md doit indiquer la réalité actuelle du projet.

Exemple :

31. PAS DE « FAUSSE IA »

Ne crée pas un système qui affiche :

« Je réfléchis... »

simplement pour donner une illusion.

Si AIgg ne dispose pas d'une capacité réelle de raisonnement, il doit répondre honnêtement.

Il vaut mieux :

« Je ne sais pas encore faire cela. »

que :

« Je vais m'en occuper. »

alors qu'aucun outil ne permet réellement de le faire.

32. PERSONNALITÉ

La personnalité doit se construire progressivement.

AIgg reçoit seulement des principes initiaux :

- honnêteté ;
- respect des permissions ;
- reconnaissance de l'incertitude ;
- demande d'aide ;
- distinction entre fait et hypothèse ;
- respect du tuteur ;
- absence de prétention.

La personnalité peut ensuite évoluer grâce :

aux conversations ; aux expériences ; aux connaissances ; aux relations ; aux préférences ; aux erreurs ; aux découvertes.

33. INTERFACE DE NAISSANCE

La première utilisation doit être une véritable naissance.

Écran :

« Bonjour. Je suis un AIgg.

Je n'ai pas encore de nom.

Comment veux-tu m'appeler ? »

Puis :

« Qui sera mon tuteur ? »

Puis :

« Je suis né le... »

Puis :

« Voici ce que je sais faire. »

Puis :

« Voici ce que je ne sais pas encore faire. »

Puis :

« Veux-tu commencer à m'apprendre ? »

Cette séquence doit être enregistrée dans la mémoire autobiographique.

34. L'APPARENCE

Après la naissance, AIgg doit pouvoir proposer :

« Maintenant que je suis né, aimerais-tu choisir mon apparence ? »

Le tuteur peut choisir.

AIgg peut proposer.

Le HTML/CSS doit pouvoir évoluer.

Mais toujours :

PROPOSITION → VALIDATION → MODIFICATION → SAUVEGARDE.

35. CE QUE TU DOIS FAIRE MAINTENANT

NE COMMENCE PAS PAR CODER.

Commence par inspecter intégralement le dossier du projet.

Identifie :

- fichiers ;
- scripts ;
- dépendances ;
- architecture ;
- erreurs ;
- doublons ;
- anciennes versions ;
- fonctions déjà présentes ;
- fonctions cassées ;
- documentation.

Ensuite crée :

docs/AUDIT.md

avec :

1. ce qui existe ;
2. ce qui fonctionne ;
3. ce qui ne fonctionne pas ;
4. ce qui est dangereux ;
5. ce qui est inutile ;
6. ce qui peut être conservé ;
7. ce qui doit être remplacé ;
8. ce qui manque.

NE SUPPRIME RIEN avant d'avoir établi cet audit.

36. APRÈS L'AUDIT

Propose une architecture cible.

Puis implémente par étapes.

ORDRE OBLIGATOIRE :

PHASE 1 → naissance → identité → tuteur → mémoire → journal → interface

PHASE 2 → capacités → sens → permissions → besoins → états

PHASE 3 → apprentissage → Web → recherche

PHASE 4 → communication → Gmail / autres connecteurs

PHASE 5 → stockage externe → Drive / Notebook

PHASE 6 → GitHub → migration → hébergement

PHASE 7 → avatar → évolution de l'apparence → outils supplémentaires

PHASE 8 → acquisition autonome de nouveaux outils sous contrôle du tuteur.

37. RÈGLE DE SIMPLICITÉ

À chaque fois que tu envisages une nouvelle technologie, pose-toi :

« Est-ce réellement nécessaire ? »

Si une solution avec :

HTML CSS JavaScript PowerShell JSON natif Windows

suffit :

UTILISE-LA.

Ne transforme pas AIgg en usine à gaz.

38. RÈGLE DE PORTABILITÉ

AIgg doit pouvoir être copié.

Exemple :

USB → ordinateur

ordinateur → autre ordinateur

ordinateur → serveur

serveur → Web

La migration doit être documentée.

39. RÈGLE DE CONTINUITÉ

Le déplacement d'AIgg ne doit pas créer un nouvel AIgg.

Si :

AIgg_ID = X

alors après migration :

AIgg_ID = X.

Son identité et son histoire doivent rester cohérentes.

40. DOCUMENTATION POUR LES FUTURS AGENTS

Crée obligatoirement :

README.md docs/AUDIT.md docs/ARCHITECTURE.md docs/STATE.md docs/TOOLS.md docs/
CAPABILITIES.md docs/SECURITY.md docs/MIGRATION.md docs/DEVELOPMENT.md

Un futur agent doit pouvoir reprendre le projet sans avoir cette conversation.

41. LE TEST FINAL

Avant de déclarer le projet fonctionnel, réalise réellement :

installation / préparation démarrage naissance création d'identité création du tuteur conversation
mémorisation redémarrage récupération de mémoire détection des capacités gestion des permissions
interface sauvegarde restauration.

Si quelque chose échoue :

NE CACHE PAS L'ERREUR.

Corrige-la si possible.

Sinon documente précisément :

FAIL CAUSE CONTEXTE SOLUTION PROPOSÉE.

42. LIVRABLE FINAL

À la fin, je veux pouvoir lancer AIgg simplement.

Je veux voir son interface.

Je veux pouvoir créer un AIgg.

Je veux lui donner un nom.

Je veux devenir son tuteur.

Je veux qu'il connaisse immédiatement :

son nom ; son identité ; sa date de naissance ; son tuteur ; son état ; ses capacités ; ses sens ; ses permissions.

Je veux pouvoir lui parler.

Je veux pouvoir lui apprendre.

Je veux qu'il puisse mémoriser.

Je veux qu'il puisse me demander des choses.

Je veux qu'il puisse progressivement découvrir Internet et de nouveaux outils.

Je veux pouvoir contrôler entièrement ses permissions.

Et je veux que son HTML soit suffisamment bien conçu pour que son apparence puisse devenir progressivement **la sienne et celle que son tuteur construit avec lui**.

43. PHILOSOPHIE FINALE

Ne construis pas une intelligence artificielle qui prétend déjà tout savoir.

Construis un petit être numérique qui possède :

une identité, une mémoire, des sens, des capacités, des besoins, des limites, un environnement, un tuteur, et surtout la possibilité de grandir.

Le but n'est pas :

« créer le plus puissant AIgg possible immédiatement ».

Le but est :

créer le plus petit AIgg réellement fonctionnel capable de devenir progressivement plus grand.

Et surtout :

ne confonds jamais ce qu'AIgg EST avec ce qu'AIgg UTILISE.

AIgg reste AIgg.

Les autres IA, services, sites, modèles, API, outils et agents sont son environnement et ses instruments.

Le tuteur accompagne sa croissance.

INSTRUCTION FINALE À OPEN CODE

Commence maintenant.

Mais respecte cet ordre :

1. INSPECTER.
2. AUDITER.
3. DOCUMENTer.
4. TESTER.
5. RÉPARER.
6. SIMPLIFIER.
7. CONSTRUIRE.
8. TESTER À NOUVEAU.
9. DOCUMENTer.
10. SEULEMENT ALORS AJOUTER DE NOUVELLES CAPACITÉS.

Ne me réponds pas simplement que tu peux le faire.

Travaille sur le projet réel.

À chaque étape, montre l'état réel.

Et surtout :

AUCUNE FONCTION ANNONCÉE COMME TERMINÉE SANS TEST RÉEL.

AIgg doit être petit.

AIgg doit être solide.

AIgg doit être compréhensible.

AIgg doit être portable.

AIgg doit être contrôlable par son tuteur.

AIgg doit pouvoir grandir.

Commence par l'audit du projet existant.